



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Análisis del modelo C4: Arquitectura de sistemas en 4 niveles

Constructos, notación, metamodelo y semántica de un lenguaje visual para describir arquitectura de software aplicado a un sistema ciberfísico.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026

¿Qué es el modelo C4?

Origen. Creado por Simon Brown entre **2006 y 2011** como respuesta a la dificultad de comunicar arquitectura con UML. Hoy es estándar de facto en la industria y cuenta con herramientas propias (*Structurizr*).

¿Cuál es su propósito? Describir la **arquitectura** de un sistema de software mediante un conjunto de **mapas jerárquicos**, cada uno dirigido a una audiencia distinta — «zoom» progresivo desde el contexto de negocio hasta el código. La estructura es el eje, pero el modelo también cubre **comportamiento** (vista dinámica) y **despliegue**.

¿Qué tipo de sistemas permite representar? Cualquier sistema de software compuesto por **unidades desplegables que se comunican**: web, microservicios, móviles e integraciones. En un **sistema ciberfísico** el firmware de cada nodo encaja como contenedor —es una unidad desplegable y ejecutable—, pero el **hardware en sí no tiene elemento propio**.

¿En qué contexto se utiliza? Documentación de arquitectura, *onboarding* de equipos, revisiones de diseño y comunicación con *stakeholders* no técnicos. Su premisa: la arquitectura no es una vista única — el contexto le importa al negocio, los contenedores a operaciones, los componentes al desarrollador.

LOS 4 NIVELES

Level 1: Context

Sistema + usuarios + sistemas externos

Level 2: Container

Unidades desplegables/ejecutables: aplicaciones y almacenes de datos

Level 3: Component

Módulos funcionales dentro de un container

Level 4: Code

Clases, funciones, métodos

Constructos y conceptos del modelo C4

CONSTRUCTO	ROL	NIVEL
Person	Usuario, actor, rol o <i>persona</i>	1
SoftwareSystem	El sistema descrito, o uno externo con el que interactúa	1
Container	Unidad desplegable y ejecutable: aplicación, almacén de datos, cola	2
Component	Agrupación lógica dentro de un contenedor	3
(nivel Code)	No hay elemento en el metamodelo: C4 remite a UML	4
Relationship	<i>description</i> , <i>technology</i> e <i>interactionStyle</i> : Sync / Async	todos

Cómo se relacionan. Los cuatro estructurales heredan de `StaticStructureElement` y se componen: `SoftwareSystem` $\blacklozenge \rightarrow$ `Container` $\blacklozenge \rightarrow$ `Component`. El nivel 4 **no llega** a la jerarquía. **Relationship** es transversal: conecta dos *Element* cualesquiera, **incluso de niveles distintos** (una *Person* usa un *Container*).

Rama aparte para despliegue: `DeploymentNode`, `ContainerInstance`, `InfrastructureNode`.

Además de los 4 niveles hay más vistas: *System Landscape* (que se sitúa **por encima** del modelo), *Dynamic* (comportamiento en ejecución, con números de secuencia) y *Deployment*.

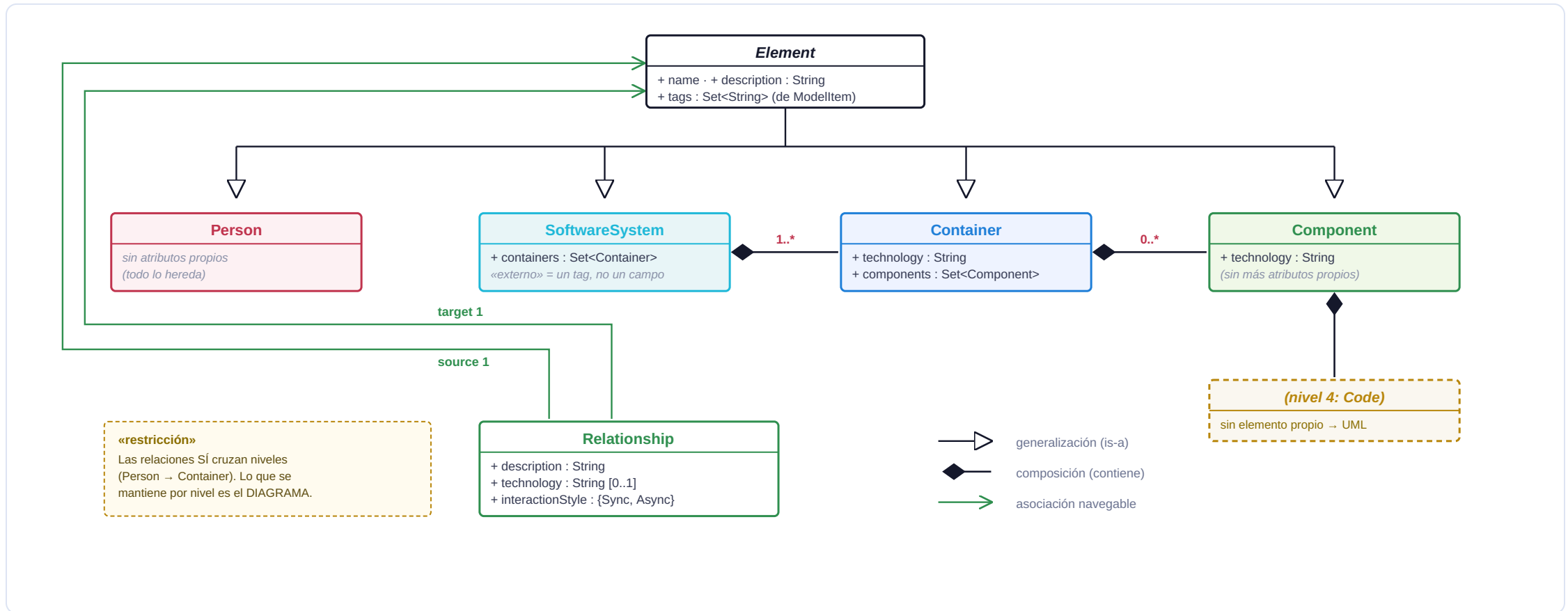
Verificado sobre `structurizr/structurizr`, paquete `com.structurizr.model` — implementación de referencia escrita por el autor de C4.

RESTRICCIONES (REGLAS DE BUENA FORMACIÓN)

- **La regla central es de alcance, no de conexión:** cada diagrama muestra **un solo nivel de zoom**. Poner componentes en un diagrama de contexto, o clases en uno de contenedores, es el error más común.
- Las **relaciones sí cruzan niveles**: una *Person* se conecta con un *Container*, y en un diagrama de contenedores aparecen las personas y sistemas externos que tocan esos contenedores.
- Un **SoftwareSystem** externo se modela como **caja negra**: no se abre aunque sepamos qué contiene. «Externo» **no es un atributo**: se marca con un *tag*, y el tag decide el estilo visual.
- Toda *Relationship* es **dirigida** y debe llevar *description*; declarar la *technology* es opcional pero recomendado.
- Todo elemento tiene **name** y **description**.

Nada de esto lo **impone** una gramática: son convenciones documentadas. Solo al escribir el modelo en *Structurizr DSL* aparecen restricciones verificables por herramienta.

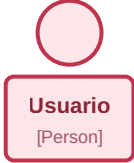
Relaciones entre constructos (diagrama de clases simplificado)



Metamodelo simplificado, derivado del código del metamodelo de referencia: `structurizr/structurizr · com.structurizr.model` (`Element`, `StaticStructureElement`, `Person`, `SoftwareSystem`, `Container`, `Component`, `Relationship`, `InteractionStyle`, `DeploymentNode`). Conceptos: Brown (2016).

Notación visual estándar de C4

Person



Usuario
[Person]

Actor humano.
Figura + etiqueta de tipo.

SoftwareSystem



SVIF
[Software System]
Control de acceso facial

Caja del sistema. Si es externo,
se dibuja en gris y punteado.

Container



Face Monitor
[Container: ESP32-CAM]
Captura e identifica rostros

Misma caja: lo que distingue el
nivel es la *etiqueta de tipo*.

Relationship



Face Monitor → Publica evento → Actuator
[MQTT]

Flecha dirigida, siempre etiquetada.
El estilo **Async** se marca con línea punteada.

C4 no impone un estándar gráfico

La forma es una *opción de estilo*: el enumerado Shape ofrece 19 (Box, Cylinder, Pipe, Person, Robot, WebBrowser...). Lo esencial no es la forma, sino los **tres datos de cada caja**: nombre, etiqueta de tipo y descripción — en el metamodelo son name, metadata y description de ElementStyle.

Formas y flags de visualización verificados en `com.structurizr.view.Shape` y `ElementStyle` (metadata, description).

¿Qué significa cada nivel? (Transformación de abstracción)

1 Context

¿Quiénes son los usuarios? ¿Qué otros sistemas existen? ¿Límites del sistema?

2 Container

¿Qué aplicaciones/nodos/DBs hay? ¿Cómo se despliegan? ¿Tecnología base?

3 Component

¿Qué módulos funcionales hay dentro? ¿Cómo se comunican? ¿Responsabilidades?

4 Code

¿Qué clases/métodos implementan? ¿Patrones de diseño? ¿Detalles técnicos?

EN SVIF:

Context: Usuario quiere acceder, cámara detecta, cerradura se abre.

Container: Dos nodos ESP32 (Face Monitor + Access Actuator) comunicados por MQTT.

Component: Dentro de Face Monitor: captura, detección, identificación, MQTT sender.

Code: `monitorearPresenciaTask()`, `capturarImagen()`, `identificarRostro()`, `publicIdentificationEvent()`.

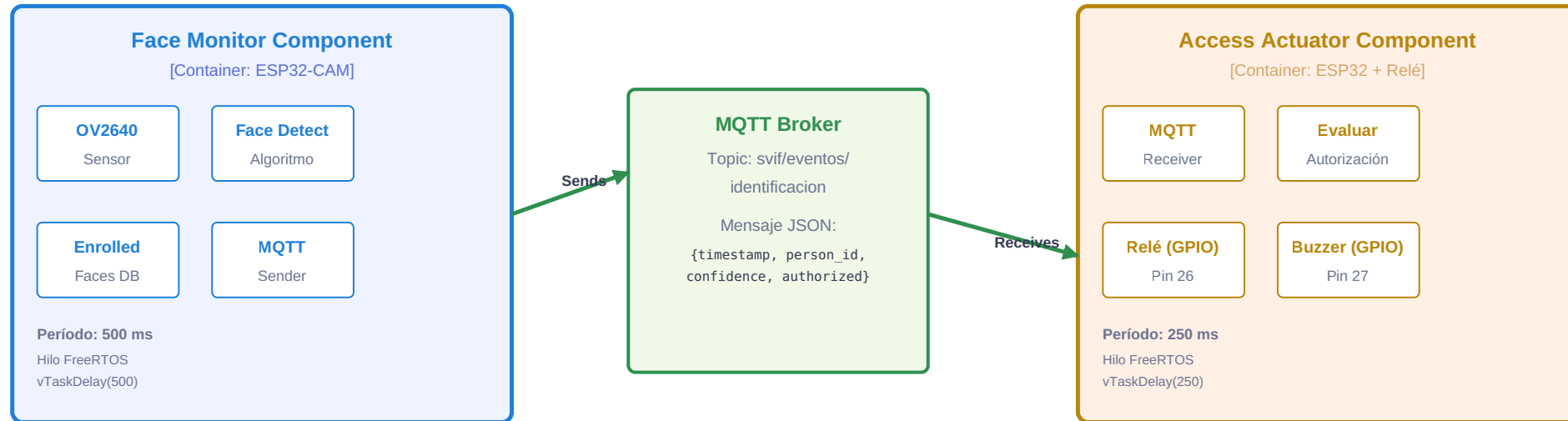
Context: Sistema en contexto



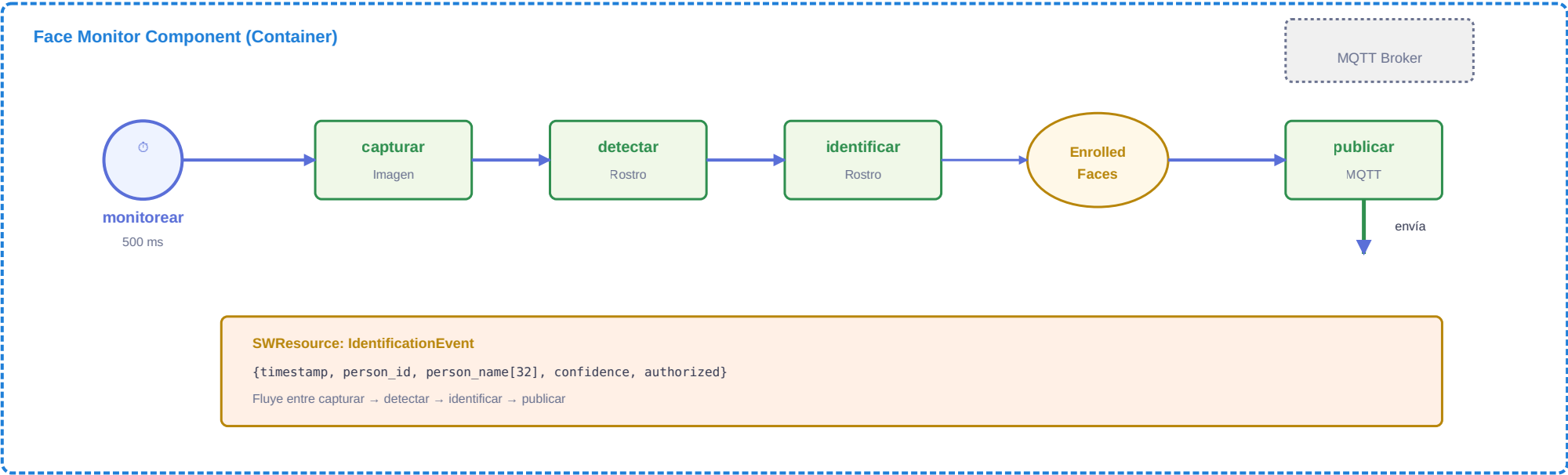
Dos cosas que mirar en este nivel.

(1) El sistema **no se abre**: los nodos ESP32 y el broker son contenedores y aparecen en el nivel 2. (2) La cerradura y la alarma son una **extensión propia**: el System Context de C4 se define sobre «personas y otros sistemas de software», y un actuador no es ninguno.

Container: Nodos y tecnología



LEVEL 3 **Component: Dentro de Face Monitor**



¿Por qué C4 es útil y cuáles son sus limitaciones para CPS?

FORTALEZAS DE C4

- **Escalable.** 4 niveles claros permiten comunicar desde CEO hasta desarrollador sin abrumar.
- **Agnóstico.** No asume tecnología; funciona igual para Java, Node, Go, IoT.
- **Composable.** Más allá de los 4 niveles: *System Landscape*, *Deployment* y *Dynamic*.
- **Sí modela comportamiento.** La *Dynamic view* ordena interacciones en tiempo de ejecución mediante números de secuencia.
- **Comunicación.** Diagramas visuales ayudan a stakeholders no técnicos a entender estructura.

LIMITACIONES PARA CPS

Precisión necesaria: C4 **sí** modela comportamiento — la *Dynamic view* ordena interacciones con números de secuencia. Lo que no alcanza es lo específico del dominio:

- **No expresa periodicidad.** La vista dinámica dice «paso 1, paso 2», no «cada **500 ms**». No hay atributo de período.
- **No expresa condicionalidad.** El OR de SVIF —desbloquear v alarmar— no tiene representación: la secuencia es lineal.
- **No expresa restricciones temporales duras.** Ni plazos ni *jitter* tolerable.
- **No tiene elemento para el mundo físico.** El *System Context* se define sobre «personas y otros sistemas de software»; un sensor o una cerradura no encajan. *DeploymentNode* modela infraestructura, no actuadores.
- **Abstracción variable.** Un *Container* abarca desde una app web hasta un ESP32 con 2 KB de RAM.

¿QUÉ ASPECTOS PODRÍAN MEJORARSE? · ¿ES ADECUADO PARA MI DOMINIO?

Mejoras que propondría. (i) Un atributo estándar de **periodicidad** en Container/Component, hoy relegado a la descripción libre. (ii) **Estereotipos de hardware** para distinguir un nodo embebido de un servicio en la nube. (iii) Un **enlace explícito a requisitos** o a objetivos, que permita justificar por qué existe cada contenedor. (iv) Reglas de validación formales: hoy la «buena formación» es convención documentada, no un metamodelo verificable —salvo que se use Structurizr DSL.

¿Adecuado para CPS? Sí, pero parcialmente. El nivel 2 describe SVIF con precisión —dos nodos y un canal MQTT— y es la vista que efectivamente usaría para explicar el sistema. Lo que no alcanza a expresar son los **500 ms** de muestreo, el pin del relé y la bifurcación *desbloquear v alarmar*. Esa carencia es justamente lo que motiva un **DSL propio para CPS**: C4 aporta contexto y contenedores; el DSL aporta comportamiento y tiempo. **Son complementarios, no competidores.**

REFERENCIAS

Fuentes consultadas

Brown, S. (2016). *Software architecture for developers. Volume 2: Visualise, document and explore your software architecture*. Leanpub. Especificación en línea: <https://c4model.com>
Language and tool for visualizing software architecture at four levels of abstraction.

Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.
Comprehensive guide to designing and implementing DSLs; discusses C4 within broader modeling context.

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). An agent-oriented model-driven development process for cyber-physical systems. *CibSE 2025* (pp. 150–164). SBC.

Process combining iStar 2.0 (CIM), DSL PIM, and C++ PSM; includes C4 for Level 2 container view.

UML 2.5.1 Specification. (2017). Object Management Group (OMG). <https://www.omg.org/spec/UML/Standard/for/unified/modeling/language/>; *C4 is positioned as simpler alternative for architecture views.*

Structurizr (s. f.). *structurizr/structurizr* — paquetes [com.structurizr.model](https://github.com/structurizr/model) y [.view](https://github.com/structurizr/view). <https://github.com/structurizr/structurizr>

Implementación de referencia del autor de C4 («remains the reference implementation»); fuente de constructos, atributos, cardinalidades, tipos de vista y notación.